LUIS ROMERO

Full-Stack Developer | AI Engineer (Node.js/TypeScript)

Mérida, Venezuela | +58 424 709 2980 | lueduar15@gmail.com | linkedin.com/in/hernandezrs955 | github.com/ROMEROLUIS15

PROFESSIONAL SUMMARY

Full-Stack Developer / AI Engineer who ships complete products: database, backend, frontend, AI and deployment. I build backend for a software company in Texas (United States), and before that I led software development for a public institution, deciding stack, architecture and quality standards. Alongside that I operate three systems in production: Cronix, my multi-tenant SaaS where AI agents for WhatsApp, voice and calls handle appointments autonomously, with a Next.js dashboard in 6 languages; a SaaS for a motorcycle workshop with Python/LangGraph agents; and an industrial CMMS that works with no connection, in daily use. My work is not plugging in an LLM: it is designing the controls that stop it from inventing data, measuring its reliability, and building the interface that makes it usable.

EXPERIENCE

Backend Developer | Complexity (Texas, United States, Remote) July 2026 - Present

Project Factory: backend of a B2B company-directory platform and of its admin panel, with a mobile app. REST API, real time over WebSockets, job queues, full-text search and AI agents (Node.js, NestJS, TypeScript, PostgreSQL, Redis/BullMQ, Elasticsearch, AWS).

- Audited the backend when I joined, presented the findings and got my remediation plan approved, which I led to completion without pausing the roadmap: through incremental refactoring, removed improper coupling between modules, cut circular dependencies from 31 to 0 and moved business logic into 330 use cases on a hexagonal architecture.
- Designed the AI agent harness I work with: numbered rules that the code itself cites, architecture and quality gates, and a check that asks for evidence behind every claim the agent makes (Orca, Gentle AI, MCP, Spec-Driven Development). With it I planned the work in 700+ tickets and delivered 1,050+ merged pull requests and 20+ production releases in three months, reviewing and approving every change.
- Built the quality and security pipeline from scratch. When I joined, no pull request ran any tests. Today every change goes through strict typing, unit, integration and end-to-end tests, security scanning (SAST/SCA, secrets) and architecture rules, and a full PR is validated in ~6 min. Unit tests grew from 20 to 1,000+ suites. The quality check dropped from 9.4 to 3.8 min through parallel execution and integration suites selected from the dependency graph.
- Led the backend of "claim your company" end to end, the product's go-to-market (GTM) program, now in production and fed by an AI-assisted loading flow I took to production: an agent builds company profiles and product drafts that nobody publishes without human approval. I also delivered the Resend integration, orders on a state machine and search on Elasticsearch.
- Designed and built an AI extraction engine with current agent-engineering practices: every value carries the quote from its source, two models must agree and code, not the model, decides what gets stored; quality is measured with evals against the human reviewer's decision and traces in Langfuse. It nearly tripled category classification precision.
- Strengthened security and operations. Took PostgreSQL row-level security (RLS) to production: from 0 to 63 enforced tables in three reversible waves, with pgTAP tests that block the merge. Resolved an urgent personal-data exposure report in under 4 hours, added rate limiting with Redis and operated AWS deployments (EC2, CloudWatch, Secrets Manager), closing two production incidents in under an hour.

IBIME Connect (Institutional platform, Remote) January 2025 - July 2026

Software Development Lead (February 2026 - July 2026) | Backend / Full-Stack Developer (January 2025 - February 2026)

- Replaced a public library network's legacy HTML/CSS system with a complete digital platform, including an AI assistant that answers citizens' queries. The institution approved it, adopted it as its official platform and, on the back of that delivery, appointed me Software Development Lead.
- Led software development for the institution: I decided the stack and architecture, set the testing and quality standards, and prioritized the roadmap together with librarians and the IT team.
- Built the React frontend on hexagonal architecture, with data validation shared between frontend and backend (Zod) so the contract cannot break silently; 420+ tests in CI (Vitest + Playwright).
- Stopped the assistant from making up answers by keeping the model out of the decision flow: a rule-based intent classifier (no LLM) routes the conversation, and if the citizen's email already exists the answer comes from the database without calling the model. A deterministic chain (intent, data protection, match threshold, review and output policy) validates before responding.
- Implemented semantic search over the institution's documents (RAG with pgvector and 768-dim Gemini embeddings, with inference on Groq gpt-oss-20b configurable per environment and Redis caching) and hardened the handling of personal data: blocked anonymous writes on sensitive tables (RLS), prevented duplicate registrations, and defined how long each piece of data is kept and how it is deleted.

Backend Developer | Kiura (Barranquilla, Colombia, Remote) April 2024 - December 2024

- Built the backend services of a teletherapy platform: paying the subscription automatically grants or suspends the patient's access (MercadoPago via webhook), with real-time chat (Socket.IO) and video sessions using per-room signed access (Agora).
- Implemented dual authentication (JWT + Google OAuth) with role-based permissions over a 26-entity domain in MySQL/Sequelize, with scheduling and clash detection; deployed on Render with the database on AWS (RDS).

PROJECTS

Founder & Full-Stack / AI Engineer | Cronix (in production; operation and maintenance) January 2026 - Present

Multi-tenant SaaS for appointment-based businesses, built and operated single-handedly (DB, serverless backend, AI, frontend and infrastructure). In production 24/7: 170+ real appointments handled by the agent and automated reminders, with no manual intervention. 1,600+ tests (Vitest, Playwright, pgTAP) and CI/CD with Sentry.

- Built the product the business owner actually sees: a Next.js 15 dashboard (App Router, Server Actions, SSR) installable as an app (PWA), with a real-time calendar, financial summary, PDF reports and 6 languages.
- Built a real-time call agent that answers over WhatsApp and books appointments in Cronix by voice (Python, Pipecat Flows, WebRTC, Deepgram): the model talks and code decides every fact and every write, including the caller's confirmation. On Cronix's side, a signed API (HMAC) with idempotency and a lock that prevents booking the same slot twice.
- Chose the call agent's model by measurement, not intuition: on the golden dataset it scored 1.00 on inventing nothing and on no unrequested writes, the first token arrives in 391 ms (p50) and every call is traced in Sentry with its latency and cost.
- Built the WhatsApp and voice agents that book, reschedule and cancel appointments on their own: the customer just asks, at any hour, and nobody at the business has to step in. Custom orchestration engine (no framework), with hard-coded routes that resolve most cases without calling the model, and restricted access to tools (Groq + Deepgram, 12 capabilities).
- Stopped the agent from inventing data or acting on the wrong customer — the real risk of giving an LLM write access to a database full of real clients — with 6 layers of control, including a mandatory two-step confirmation before any write and a second AI model that reviews the operation and can veto it (LLM-as-judge).
- Built the system that checks whether the agent answers well: I saved real conversations as a benchmark and replay them on every change; CI blocks the merge if the agent gets worse (golden conversations), and I also measure it with DeepEval and an LLM judge (relevancy and hallucination). Every conversation is logged with its latency (p50/p95) per business in a custom dashboard (PostgreSQL + LangSmith), across 9 Edge Functions (Deno).
- Audited my own security until I found a hole in it: I discovered that 13 database functions let one business read another's data, reproduced the attack to confirm it, closed it with an ownership check, and locked it down with 147 automated security tests (pgTAP over 59 RLS isolation policies).
- Integrated 4 payment methods (PayPal, NOWPayments, Pago Movil, Binance) that cannot double-charge: even if the provider notifies the same payment twice, the business is billed once.

Motosmax Cordialidad | Full-Stack Developer (Barranquilla, Colombia, Remote) (delivered in phases; in production with support) June 2026 - Present

- Shipped the SaaS a motorcycle workshop runs on every day: work orders with a state machine, per-branch inventory, versioned quotes with client approval and PDF, motorcycle sales with a payment plan, PDF contract and a sales dashboard, and customer records. Monorepo with a Next.js 15.5 / React 19 frontend (installable as an app) and a NestJS API (hexagonal architecture, 37 Prisma data models).
- Automated both customer service and back-office queries with a Python microservice (FastAPI + LangGraph): one agent serves the workshop's customers over WhatsApp and another assists the back office with session memory, periodic reports and low-stock alerts. The LLM model (Groq/DeepSeek) is configurable per environment, and the WhatsApp channel handles Meta's 24h window and surfaces send failures.
- Locked down service-to-service communication with credentials that expire in 5 minutes and encryption of sensitive fields; job queues (BullMQ/Redis), photo/PDF storage on Cloudflare R2 and a WebSocket gateway for real time. 400+ tests (Jest, Vitest, pytest + Playwright E2E) in CI. Every change starts from a versioned spec (Spec-Driven Development).
- Found and closed a privilege escalation in production: a read-only role could change the workshop's configuration (the VAT rate and the data printed on quotes and contracts); I reproduced it, fixed it with per-role permissions, and pinned it with a test that stops it from coming back. I also moved rate limiting to key by user identity rather than shared IP.

CMMS Hidrobombas Merida (Client, Remote) | Full-Stack Developer (delivered in phases; in production with support) April 2026 - Present

- Got an industrial maintenance company off paper: I built the CMMS (52 REST endpoints) that replaced their printed inspection sheet and is used every day, with requirements gathered directly with the client. I measured its real ceiling before promising anything: with 200 concurrent users (k6) it holds with zero server errors (p95 276 ms), and the limit I found — reports are numbered one at a time so numbers never repeat — I chose not to touch, because removing that lock would bring back duplicate report numbers and the ceiling sits far above the client's real usage.
- Delivered the app the technician files reports with in the field, even with no signal: React 19 saves the work on the device and syncs it once the connection is back, with a 13-step guided form (mains voltage, pressure switch, noise in dB, among others), on-screen signature and a PDF at the end; sessions with refresh and remote revocation (JWT).

Advanced Product Search API (The technical challenge that got me into Complexity; deployed and public) | Backend Developer July 2026

Product/manufacturer search API with relevance ranking, filtering, faceting, autocomplete and query suggestions.
Live: advanced-search-api-chet.onrender.com/docs · github.com/ROMEROLUIS15/advanced-search-api

- Built the search that ranks results by what the person is actually looking for rather than by insertion order: BM25 relevance on Elasticsearch 8 with typo tolerance, adjusted by popularity and how recent the product is (function_score), with combinable filters, type-ahead autocomplete and "did you mean" suggestions.
- Solved faceted filtering without locking the user into their own selection: each dimension's counts are computed excluding its own filter, so the user can widen the search instead of hitting zero results; results, facets and suggestions come back in a single round-trip. Hexagonal architecture (framework-free domain, Elasticsearch and Redis behind ports) a cache that never fails a request if Redis goes down, and an API key on every client endpoint. Under a flood the rate limit cuts exactly at the budget and returns not a single server error.

- Delivered it deployed and verified, not "works on my machine": Docker + Render/Elastic Cloud with a public OpenAPI contract, 500+ tests (unit, e2e and integration against real Elasticsearch, 97% coverage), a k6 load test (398,000 requests, 0 failures) and automated security in CI over the code, the dependencies, the secrets in git history, the shipped image and the running API (CodeQL, gitleaks, Trivy + SBOM, OWASP ZAP: 128 URLs, 0 findings). Index schema changes go live without taking the API out of service, by moving an alias atomically.

TECHNICAL SKILLS

Frontend: React, Next.js 15, App Router, React Server Components, Server Actions, SSR, Tailwind CSS, Radix UI, TanStack Query, Zod, PWA, Service Workers, Offline-First, WebSockets, i18n

Backend: Node.js, TypeScript, JavaScript, Python, NestJS, Express, FastAPI, Pydantic, asyncio, uv, REST APIs, OpenAPI/Swagger, Transactional email (Resend, React Email), Socket.IO, Serverless, Edge Functions, Deno, Microservices, Event-Driven Architecture, Domain Events, Unit of Work, Webhooks, Idempotency, SOLID, Refactoring, Technical Debt, Dependency Injection, Hexagonal Architecture, Clean Architecture, Architecture Decision Records (ADRs), SSE, C#/.NET (basic)

AI: LLM Integration, LangChain, LangGraph, AI Agents, Multi-Agent Orchestration, Tool Calling / Function Calling, Structured Outputs, Prompt Engineering, Agentic AI, Vercel AI SDK, RAG (Retrieval-Augmented Generation), Semantic Search, Vector Databases (pgvector), Anti-Hallucination, AI Guardrails, LLM Evaluation (DeepEval), LLM Observability (LangSmith, Langfuse), MCP, Voice AI, Pipecat, Pipecat Flows, WebRTC, Speech-to-Text, Text-to-Speech, OpenRouter, OpenAI API, Groq, DeepSeek, Gemini, Deepgram, n8n

Harness Engineering & AI-assisted development: Claude Code (Anthropic), Codex (OpenAI), Orca (ADE), Gentle AI, Engram (persistent agent memory), MCP (Linear, Sentry, AWS, Resend, Figma), multi-agent orchestration, subagents, agent hooks and skills, Context Engineering

Data: PostgreSQL, SQL, Prisma, TypeORM, Supabase, Neon, AWS (RDS), MySQL, Sequelize, SQLite, Redis, BullMQ, pgvector, Elasticsearch, Full-Text Search & Relevance (BM25), Faceting, Row-Level Security, Multi-tenant

Security & payments: JWT, OAuth 2.0, WebAuthn/Passkeys, RBAC, Rate Limiting, HMAC, Threat Modeling, Security Auditing, PayPal, MercadoPago, QStash

DevOps & testing: Docker, AWS (S3, EC2 Auto Scaling, ECR, CloudWatch, Secrets Manager), Git, GitHub Actions, Husky, Linear, CI/CD (Continuous Integration / Continuous Deployment), Vercel, Render, Cloudflare (Pages/R2), Sentry, Observability, OpenTelemetry, SAST/DAST/SCA (CodeQL, Semgrep, OWASP ZAP, Trivy, gitleaks, Dependabot), SBOM, Load Testing (k6), Vitest, Jest, pytest, Hypothesis (property-based testing), Snapshot testing, mypy (strict), Ruff, Bandit, pip-audit, Playwright, pgTAP, Spec-Driven Development (SDD)

EDUCATION

Computer Systems Engineering | UPTM Kleber Ramirez - Graduated: Oct 2026

Full-Stack Bootcamp | Academlo - Mar 2024

LANGUAGES

Spanish: Native | English: Intermediate (B1+), advanced technical reading and writing